

Electronic Systems Lab Report

Group 2 - Odometry

1st Viet Hung Tran
Matriculation number: 229050
Hochschule Heilbronn

2nd Quoc An Nguyen
Matriculation number: 229040
Hochschule Heilbronn

Abstract—This paper documents the process of building a Raspberry Pi based robot car with odometry logging features. Utilizing a Robot Operating System 2 (ROS2) network, the system establishes a reliable wireless teleoperation framework and locational mapping. The hardware architecture integrates a drive chassis, infrared wheel encoders for tracking wheel rotation, and a PMW3901 optical flow sensor for monitoring relative surface movement. To enhance positional accuracy, a custom data fusion node was developed to cross-correct sensor streams, specifically discarding erroneous encoder readings during instances of physical obstruction or wheel slip. The robot's spatial awareness—comprising its 2D coordinates and yaw angle—is continuously calculated using differential kinematics and displayed through a live graphical user interface on the operator's machine. Experimental evaluations confirm the functional viability of the ROS2 teleoperation and tracking network, while also identifying mechanical drift and accumulative rotational errors as primary limitations of the prototype's open-loop design.

I. INTRODUCTION

Today in the field of Automotive Engineering, odometry-based features have become more and more prevalent. The knowledge of the vehicle's current orientation and position work in tandem with advanced sensors laid the foundation for modern functions such as autonomous driving, precision parking, etc. As the smart automotive industry advances, the requirement for environmental awareness leads to more forms of odometry being introduced, with two of the most dominant form being Wheel Odometry and Image Odometry.

For such advanced systems, it is mandatory that a form of microcontroller (MCU) or central processing unit (CPU) to be implemented and fine-tuned per system basis. The establishment of a communication interface is also required to ensure stable and efficient coordination between components. With the goal of simulating and researching automotive smart functions, our group project implemented basic odometry features for a Raspberry Pi based car chassis. A combination of wheels encoders and optical sensor ensure the locational awareness for the operator. Low-latency and wireless control is provided by a ROS2 network serving as the backbone of the system.

This paper serves as both a report and documentation of our work. It will discuss the theories, the implementation process and the difficulties faced during development. Finally, a complete product is presented with evaluations on both strengths and weaknesses.

The remainder of this report is structured as follows: Section II outlines the technical background and the foundational technologies utilized in our design. Section III details the conceptual approach and mathematical models governing the odometry system, while Section IV covers the physical hardware implementation. The system's software deployment is divided into two parts: Section V discusses the operating system initialization, and Section VI explains the ROS 2 network and node configurations. Section VII presents the experimental results and testing observations. Finally, Section VIII offers a discussion and outlook on system performance, followed by reflections on team contributions in Section IX and core learning outcomes in Section X.

II. TECHNICAL BACKGROUNDS

A. Robot Operating System 2 (ROS2)

The Robot Operating System (ROS) is a set of software libraries and tools that help you build robot applications. It has long being considered the golden standard in robotics and automation. Renowned for consistency, low latency and standardized interface, complex ROS networks spanning thousands of nodes, topics and components formed the framework of many robotics systems globally.

Our project utilizes ROS2 Humble Hawksbill for ROS network deployment. As a long term support (LTS) version made specifically for Ubuntu 22.04 operating system, this improves consistency across multiple devices for seamless collaboration between team members.

The major building block of ROS 2 is a node. A node is an independent program or service that handles a specific task. ROS 2 lets all these nodes talk together by allowing them to send and receive messages between each other. [2]. Data in a ROS2 network can move between nodes either by topics, services or actions. In case of this projects, we used topics as our main method of communication.

A Topic is a data stream used to exchange information between nodes. Topics are used to send frequent messages of one type, such as sensor readouts or information on motor goal speed. Each topic is registered under a unique name and with a defined message type. Nodes can connect to the topic to either publish messages or subscribe to them. There are no restrictions on the number of different nodes publishing or subscribing. [3] A package in ROS 2 is a set of files (a folder), that contains all the things needed to achieve a certain

functionality in a robot. A package ideally would contain, multiple nodes, a launch file, message definitions, and other things libraries needed to achieve desired functionalities. [2]

B. Secure Shell Protocol (SSH Protocol) and Open-SSH

The SSH Protocol is a cryptographic network protocol for operating network services securely over an unsecured network. Its most notable applications are remote login and command-line execution.

SSH uses the client-server model. An SSH client program is typically used for establishing connections to an SSH daemon, such as sshd, accepting remote connections.

On Unix-like systems, the list of authorized public keys is typically stored in the home directory of the user that is allowed to log in remotely, in the file `~/.ssh/authorized_keys`. This file is respected by SSH only if it is not writable by anything apart from the owner and root. When the public key is present on the remote end, and the matching private key is present on the local end, typing in the password is no longer required. However, for additional security, the private key itself can be locked with a passphrase.

C. Raspberry Pi 4

The Raspberry Pi 4 is a small single-board computers (SBCs) originally developed in the United Kingdom by the Raspberry Pi Foundation in collaboration with Broadcom. One dominant feature of this device is the included general-purpose input/output (GPIO) headers in the form of a 40-pin connector. These allow the SBC to be connected to multiple electronic components crucial for our odometry project.

By installing an instance of Ubuntu 22.04 on the Raspberry PI, we have create an environment both to receive data from sensors via physical connectors and to host the required ROS nodes for data transmission.

D. Wheel Encoders

The wheel encoders in our projects consist of two separate components: the encoder disk and the IR slot sensor. These components are illustrated in figure 1:

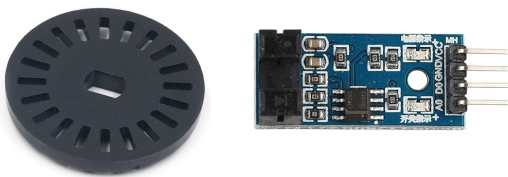


Fig. 1. From left to right: encoder disk [8] and IR slot sensor [9]

The encoder disk consists of 20 slots spreading evenly around the circumference. The middle slot is connected to the motor shaft so that the disk rotates every time the motor turns.

The slot sensors contains an IR emitter and receiver with a

small space in between. Every time the receiver status updates (from receiving to not receiving the IR light and vice versa), the device sends an electrical pulse through the digital output (D0) pin. Another function of this device is deriving the disk rotation speed from the intensity of IR light. By using the analog output (A0), we can get a continuous stream of values representing the intensity, but this function will not be considered for our implementation.

E. Robot Chassis

The provided package includes a mica frame for component mounting, two brushless motors and wheels and a caster wheel for system balancing. A L298 Dual full-bridge driver was also provided, represented by the following diagrams:

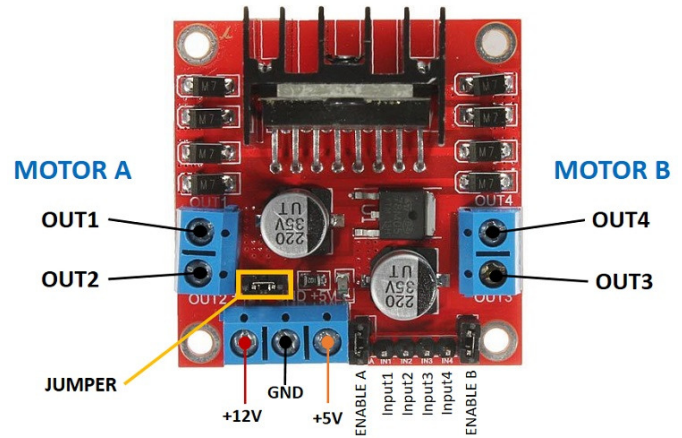


Fig. 2. L298 Motor Driver with ports labeled [4]

Direction	Input 1	Input 2	Enable 1
Forward	0	1	1
Backwards	1	0	1
Stop	0	0	0

Fig. 3. Port logic for one motor [4]

By sending PWM signals to the two enable pins (ena and enb), the speed of the motor can be adjusted. The motor speed is proportional to the duty cycle. However, the motors might not spin for small duty cycles. In our case, 0.4 seems to be the minimum value to ensure operation.

F. PMW3901 Optical Flow Sensor

The PMW3901 is an optical flow sensor that detects surface movement using an integrated optical flow algorithm. It works similar to a computer mouse sensor, tracking relative motion by comparing image patterns captured by its tiny camera. The PMW3901 computes the flow internally and provides a difference in pixels between each frame. The sensor itself does not provide absolute position data but rather the rate of movement in X and Y directions. [5]

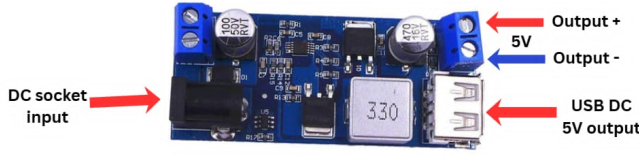


Fig. 4. DC-DC converter (Adapted from [7])

G. Power

A 2200mAh LiPo battery powers every components on the chassis. As the battery voltage (8.4V) is significantly higher than the rated voltage of other components (5V), a DC-DC converter is used to step down the potential as well as to provide an USB connector interface to power the Raspberry Pi. The 2-pin output can be utilized to power the motors and the motor driver circuit. The connections can be visualized by the figure below:

III. APPROACH

A. To achieve a remote-controlled robot

In order to fully control the chassis using a ROS network, it is mandatory that the controller host and the Raspberry Pi SBC to be connected to the same local network. We made use of the *Mobile Hotspot* feature on windows 11 and configured the Raspberry Pi to automatically connect to the hotspot. By setting up a virtual machine running Ubuntu 22.04 with a bridged network adapter, we have set up an environments where nodes from both systems are discoverable and be able to link up.

For a simple remote-controlled setup, we need one control publisher node hosted on the controller and another subscriber node on the robot's system that takes the transmitted data from the controller and send directive commands to the motors.

1) *Controller node*: The directive inputs are mapped to the keyboards as follows: 'w' for forwards, 's' for backwards, 'a' to turn left and 'd' to turn right. These commands are given every time a corresponding keys is held down and should be cancelled immediately in case the key is released. Due to an operating system limitation, there exists a noticeable interruption of the key stream when the user holds down a key on the keyboard. This causes a 'jolt' every time a directive command is given to the car. Another solution is to listen for individual key presses instead of key streams. While being a simple solution for our problem, this method oversimplifies the control so that features such as throttle dampening or combined inputs. We settle for another solution, in which we programmed the control node to ignore the first few inputs,

only transmitting the later continuous stream. The drawback of this method is it forces the operator to account for a slight delay from input to execution.

The key stream representing the control input is then published onto a topic, where it will be delivered to the motor node.

2) *Motor node*: The motor node is programmed to pulled the keystream from the topic previously mentioned, convert them into signals based on the L298 motor driver logic, then sent out said logic as digital signal through the Raspberry Pi GPIO pins.

One major issue for this node is the sideways drift from both the power mismatch between the two motors and the caster wheel's resistance. We partially alleviated this problem by manually fine-tuning the motors using PWM inputs. The remaining drift is considered hardware limitation and shall be ignored.

B. Odometry - Mathematical approach

1) *Yaw angle measurement*: To calculate this value, certain parameters must be known and manually input, namely the wheel track (L), wheels diameter (d) and the encoder tick per revolution. The distance traveled per encoder tick can be derived as:

$$M = \frac{\pi d}{ticks_per_revolution} \quad (1)$$

in which M has the unit of meters/ticks and *ticks_per_revolution* is set to 20 as previously discussed. As the two wheels spin counteractively during turning, the angle change rate in rad/tick is calculated as follows:

$$\alpha = \frac{1}{2} \frac{360M}{L\pi} \frac{\pi}{180} = \frac{M}{L} \quad (2)$$

From there, the yaw angle can be calculated as the angle resulted from the tick differences between two wheels:

$$\psi = \alpha(right_ticks - left_ticks) \quad (3)$$

From the equation, we can see that when the robot is left oriented, ψ is a positive value in radian while a right oriented situation results in a negative yaw angle.

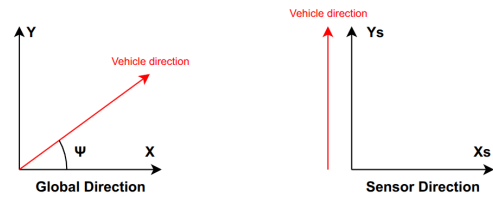


Fig. 5. Global coordinate and PMW3901 reported values

2) *Directional vector construction*: The PMW3901 node returns the values of Δy_s and Δx_s as the vertical and horizontal pixel shift in time Δt without accounting for the orientation. In order to represent both the relative coordinate and the orientation at the center point of the chassis, We must map the given information to a global coordinate. The

formulas for infinitesimal change in global x and y are given as:

$$dx = dy_S \cos \psi + dx_S \sin \psi \quad (4)$$

$$dy = dy_S \sin \psi - dx_S \cos \psi \quad (5)$$

Thus, the changes represented in global coordinates can be derived:

$$\Delta X_{global} = \Delta y_S \cos \psi + \Delta x_S \sin \psi \quad (6)$$

$$\Delta Y_{global} = \Delta y_S \sin \psi - \Delta x_S \cos \psi \quad (7)$$

As the above data consider the shift in discrete time Δt , the current position can be calculated by adding up all of the changes from previous reports:

$$X_{current} = X_{previous} + \Delta X_{global} \quad (8)$$

$$Y_{current} = Y_{previous} + \Delta Y_{global} \quad (9)$$

Where $X_{previous}$ and $Y_{previous}$ denote the shift of X and Y in pixels since sensor initialization, at the sensor's point of reference.

3) *Sensor offset compensation*: The PWM3901 was installed at the front of the chassis. Therefore, an offset have to be made to report the true location of the robot and consider its orientation, as per illustrated in figure 5.

With L_s representing the distance between the car center point and the sensor mounting point:

$$X_{center} = X_f - L_s \cos \psi \quad (10)$$

$$Y_{center} = Y_f - L_s \sin \psi \quad (11)$$

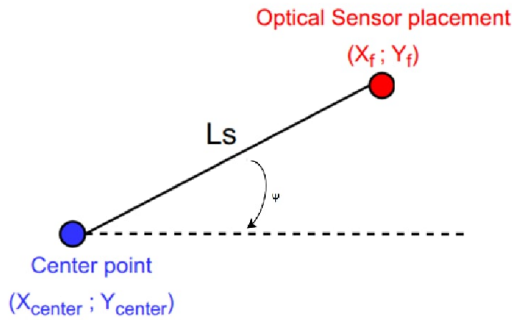


Fig. 6. Visualization of sensor offset

C. Odometry - Node design

1) *Optical sensor node*: This node governs the operation of the PMW3901 optical sensor. As the measurement performed by this sensor is highly more accurate compared to the wheels encoder, we will implement it to capture position-related data. Moreover, since this sensor updates data only if the chassis moves, it can be used to correct recorded data from the encoder in scenarios where the motors spin but the chassis got stuck.

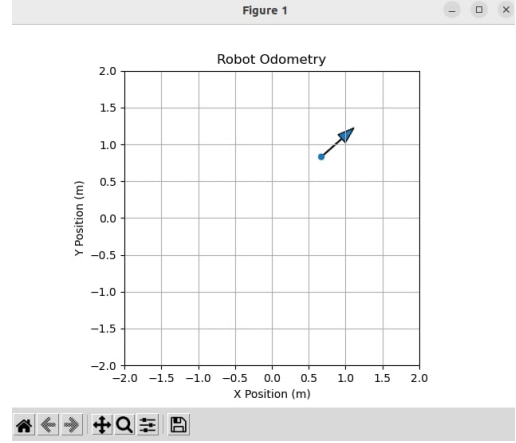


Fig. 7. Output graph of the Display Node

The data recorded from this sensor should then be processed in a data fusion node before reaching the operator.

2) *Wheel IR slot encoders*: With two encoders encoding the movements of both motors, this node is important for later orientation-related calculations. Two GPIO pins record electrical pulses sent from the two encoders, get added up to form two sum values, and finally sent to the data fusion node.

3) *Data fusion node*: The data fusion node receives data from the two sensor nodes and calculate the odometry values to display, namely the chassis's 2D coordinate represented by x and y, and the yaw angle (the heading of the robot). The working principles behind this node requires specifications such as the wheel diameter, spacing between the wheels and the number of slots on the encoder disk to be manually measured and input. Derived results would then be sent to the display node for visualization.

4) *Odometry node*: This node is hosted on the operator's device. Its main purpose is to show the data processed by the fusion node to the operator. After a set time interval, the display of data will be refreshed with information being visualized in two ways:

- Numerical: The node outputs the x, y and yaw value directly into the terminal.
- Graphical: Using the *matplotlib* python package to generate a plot with x and y being the two axes. A dot is drawn on the display representing the current coordinate of the car. A vector arrow is then drawn with the base at the mentioned dot, showing the chassis's current orientation

D. Sensor cross-correction in cases of abnormality

This feature would be integrated into the data fusion node. We specifically check for cases where the wheel encoders update their reading, but data from the optical sensor shows no changes. In such cases we discard the encoder reading entirely. The reverse can also be applied to account for optical sensor drift.

IV. HARDWARE IMPLEMENTATION

The full wiring diagram can be found at Appendix A.

A. L298 connections

1) Connections from Raspberry Pi:

- **GPIO 24** to **In4**
- **GPIO 25** to **In3**
- **GPIO 27** to **In2**
- **GPIO 22** to **In1**
- **GPIO 13** to **Enb**: for left motor PWM
- **GPIO 18** to **Ena**: for right motor PWM
- 5V pin to +5V port with grounding acts as a signal line.

Furthermore, a connection to the DC-DC converter through the +12V port provides the necessary power to drive the motors.

2) *Connections to motors*: The 4 output ports on the L298 are wired to their specific motors, with In4 and In3 governing the left motor, In2 and In1 for the right motor.

B. Sensor wirings

1) *PWM3901*: The sensor is connected to the Raspberry Pi as follows:

- **MISO** to **GPIO 9**
- **CS** to **GPIO 8**
- **SCK** to **GPIO 11**
- **MOSI** to **GPIO 10**
- **INT** to **GPIO 19**
- 3.3V pin (Board pin 1) as power source and GND for ground.

2) *Slot IR encoder*: : Beside the power connections, the sensor readings is transmitted with these connections:

- Left sensor: **D0** to **GPIO 17**
- Right sensor: **D0** to **GPIO 23**

V. OS INITIALIZATION

A. Raspberry PI

We flash an instance of Ubuntu 22 LTS on the provided sd card. After the initial setup, we proceed to install and set up open-ssh on the operating system using the following commands:

- `sudo apt install openssh-server`
- `sudo systemctl enable ssh`
- `sudo systemctl start ssh`

The Ubuntu Firewall (ufw) by default blocks ssh connections as well as prevents local ROS nodes communicating with outside nodes. For this project only, we disable the firewall completely by running `sudo ufw disable` in the terminal. To install ROS2 Humble, we follow the instruction from the official documentation [1] through the following steps:

- Set locale.
- Set up sources and installation of ROS2 packages.
- Environment setup.

additionally, we can add the ROS2 source to the bash config file so we don't need to manually type the command on terminal start-up. We edit the `bashrc` file using `sudo nano ~/.bashrc` and add this line to the end of the file: `source install/setup.bash`.

An automatic hotspot connection to the host machine is also

important for working without a display for the Raspberry PI. This can be done by only saving the credentials of the desired hotspot.

B. Host machine (operator machine)

Our host machine primary operating system is Microsoft Windows 11. The hotspot feature on this operating system allows the user to see the ip address of connected devices. Therefore, it is not required for us to set up a static IP on the Raspberry PI.

However, with ROS2 Humble main operating system being Ubuntu 22.04, we have to set up a virtual machine (VM) to run the ROS nodes responsible for sending control input and displaying the result. For our case, we set up our VM using Oracle VirtualBox. For the virtual machine to be able to communicate with the Raspberry Pi over hotspot, we have to use a bridged adapter for networking. This bypasses the host's routing, making the VM appear as an independent physical device with its own IP address on the hotspot. The setup process for ROS2 is similar to the steps taken on the Raspberry PI.

VI. SOFTWARE CONFIGURATION

A. ROS2 network deployment

ROS nodes are contained inside a ROS workspace. A workspace is a directory that stores ROS2 package together with the build, installation, and log files generated during compilation. Our network requires the configuration of two distinct ROS workspaces. one on the host machine and the other on the Raspberry PI. Their configuration steps however, are mostly similar.

To create a ROS2 workspace on an Linux machine, we need to ensure the directory `"/<project_name>/src"` exists. Then run the following command while inside the `src` directory:

```
ros2 pkg create my_robot --build-type
ament_python --dependencies rclpy std_msgs
```

with `my_robot` being the name of our package.

With our workspace created, we can compile our package using `colcon build`. The workspace should be rebuilt whenever the nodes code is modified.

The Python source files for each node shall be placed inside the package directory (alongside `__init__.py`). For our project, we created `odometry_node.py` and `control_node.py` for the operator device, while the rest are created inside the workspace of the Raspberry Pi.


```

ros2_ws/
├── src/
│   ├── my_robot/
│   │   ├── package.xml
│   │   ├── setup.py
│   │   ├── resource/
│   │   └── my_robot/
│   │       ├── __init__.py
│   │       ├── odometry_node.py
│   │       └── control_node.py

```

Fig. 8. Workspace's directory tree on the operator machine

Before the nodes can be executed using the `ros2 run` command, they must be registered in the package's `setup.py` file. This is achieved by adding each node to the `console_scripts` section right under `entry_points`, as shown in figure 8. Afterward, the same procedure is applied for setting up ROS2 workspace on the Raspberry PI, for the `encoder_node`, `optical_flow_node`, `motor_node` and `fusion_node`.

```

entry_points={
    'console_scripts': [
        'odometry_node = my_robot.odometry_node:main',
        'control_node = my_robot.control_node:main'
    ],
}

```

Fig. 9. entry_point section, setup.py

B. Network overview

The components and connectivity of the ROS2 network for this project is illustrated by the following RQT Graph:

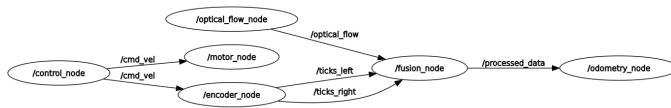


Fig. 10. RQT Graph for the ROS network

In this design, the control node and odometry node is hosted on the operator's device to send directive commands and display the results, respectively. The remaining nodes runs on the PI's hardware to receive the input, process the data, and send back the processed results.

C. Control_Node.py

This ROS 2 Python script implements a Keyboard Teleoperation Node. Its primary purpose is to capture keyboard inputs (W, A, S, D) from the terminal in real-time and translate them into velocity commands (`geometry_msgs/msg/Twist`) to drive a robot via the `cmd_vel` topic. It imports low-level Unix terminal control modules (`termios`, `tty`, `select`) to alter how the terminal handles input.

1) Initialization:

- **`uper().__init__('control_node')`**: Initializes the underlying ROS 2 Node base class and names this specific node `control_node`.
- **`self.create_publisher(...)`**: Configures a ROS 2 publisher.
 - Message Type: `Twist` (used universally in ROS for representing linear and angular velocities).
 - Topic Name: `'cmd_vel'` (Velocity Commands).
 - Queue Size: 10 (the history depth for holding messages).

```

class ControlNode(Node):
    def __init__(self):
        super().__init__('control_node')
        self.publisher = self.create_publisher(Twist, 'cmd_vel', 10)

```

Fig. 11. Initialization function

2) **`get_key` function**: This function safely extracts a single character from the keyboard while bypassing the holdkey limitation of the operating system.

- **`tty.setraw(...)`**: Shifts the terminal into "raw mode." Keystrokes are sent to the script immediately as they happen, bypassed by normal processing (like line buffering or echoing characters to the screen).
- **`select.select(...)`**: Acts as a non-blocking timer. It checks if `sys.stdin` (the keyboard stream) has any characters ready to read. It will wait for a maximum of 0.25 seconds.
 - If a key is pressed within 0.25s, `rlist` evaluates to `True`, and `sys.stdin.read(1)` grabs that single character.
 - If no key is pressed, it times out, and key defaults to an empty string `''`.
- **`termios.tcsetattr(...)`**: Immediately resets the terminal back to its original settings (settings) right after reading, ensuring the environment remains stable.

```

def get_key(self):
    tty.setraw(sys.stdin.fileno())
    rlist, _, _ = select.select([sys.stdin], [], [], 0.25)
    if rlist:
        key = sys.stdin.read(1)
    else:
        key = ''
    termios.tcsetattr(sys.stdin, termios.TCSADRAIN, settings)
    return key

```

Fig. 12. get_key function

3) **`run` function**: The **`run`** function executes a `while True:` loop that handles the translation of keys into actual robot movements. Inside the loop, a blank `Twist` message is created on every iteration. By default, all values inside it (`linear.x`, `linear.y`, `linear.z`, `angular.x`, `angular.y`, `angular.z`) are initialized to 0.0. The logic then maps the captured key to the `Twist` message fields:

Key Pressed	Action	Code Mapping
w	Move Forward	msg.linear.x = 1.0
s	Move Backward	msg.linear.x = -1.0
a	Turn Left	msg.angular.z = 1.0
d	Turn Right	msg.angular.z = -1.0
Ctrl+C	-	KeyboardInterrupt
Other/None	Stop	msg.linear.(x/z) = 0.0

The final step is to broadcast the signal and log the data:

```
self.publisher.publish(msg)
self.get_logger().info(f'Linear: {msg.linear.x}, Angular: {msg.angular.z}')
```

Fig. 13. Publish section - Control Node

where:

- **publish(msg)**: Broadcasts the configured velocity message over the network to the `cmd_vel` topic.
- **get_logger().info(...)**: Prints the currently sent speeds directly to your console window for easy debugging.

D. Motor_Node.py

This node listens to the velocity commands (the `cmd_vel` topic) generated by the control node, and translates those virtual commands into physical electrical signals to drive two DC motors via the Raspberry Pi 4 and the L298N motor controller. The Python code utilizes `gpiozero` library, which is a Python library for controlling Raspberry Pi GPIO pins.

1) *Initialization*: We use this function to define used pins, the PWM compensation factor and subscription to the topic `cmd_vel`.

- PWM Pins (`ena`, `enb`): Configured using PWMLED. PWM (Pulse Width Modulation) is used to control the speed of the motors by rapidly turning the power on and off. ENA controls the right motor's speed and is mapped to GPIO18, while ENB controls the left motor's speed at GPIO13 pin.
- The Compensation Factor (`self.compensate = 0.23`): Out of the two motors we received from the chassis kit, one spins slightly faster than the other, causing the robot to drift instead of driving straight. This feature utilized PWM to subtracts 23% power from the right motor (`ena`) to keep the robot driving in a straight line.
- Direction Pins (`in1` to `in4`; mapped as GPIO 22, 27, 25 and 24 respectively): Configured as standard digital outputs (LED). These control the H-bridge inside the motor driver, determining the direction the motor spins (forward or backward).
- The Subscriber: `self.create_subscription(...)` tells ROS 2 to listen to the '`cmd_vel`' topic and pass any incoming messages to the `self.callback` function.

2) *callback function*: When a velocity message arrives, this function extracts `linear.x` (forward/backward intent) and `angular.z` (left/right turning intent). The control logic is represented by the table below:

Action	ena - enb	in1 - in2	in3 - in4
FORWARD	1 - 0.77	OFF - ON	OFF - ON
BACKWARD	1 - 0.77	ON - OFF	ON - OFF
LEFT	0.6 - 0.6	OFF - ON	ON - OFF
RIGHT	0.6 - 0.6	ON - OFF	OFF - ON
STOP	0 - 0	OFF - OFF	OFF - OFF

E. Encoder_Node.py

This node primary purpose is to read signals from physical wheel encoders, calculate how many "ticks" (rotations or partial rotations) each wheel has completed, and publish that data for other nodes to use (like an odometry or motor control node).

Because our IR encoders cannot detect the direction a wheel is spinning on their own, this node also listens to the robot's movement commands to guess whether it should be adding or subtracting ticks.

The accumulated ticks of each wheels shall be published as two Int32 variables with a time interval of 0.1 seconds

1) *Initialization*: This section defines the publishers and subscriber of the function, while also initializes the sensors and parameters required for counting.

- Publishers: It sets up two publishers (`pub_left` and `pub_right`) to send out Int32 messages on the `ticks_left` and `ticks_right` topics. These messages have a queue size of 10.
- Subscriber: It creates a subscriber to listen to the `cmd_vel` topic, which carries `geometry_msgs/Twist` messages containing velocity commands. Whenever a message is received, it triggers the `self.getdirection` callback function.
- Sensor Setup and Pin Configuration: The node uses the `gpiozero` library to initialize `sensor_left` on GPIO pin 17 and `sensor_right` on GPIO pin 23. Both sensors are configured with floating pin (`pull_up=None`) and rising-edge activation (`active_state=True`) as per the IR encoder specification.
- Interrupts: The node makes use of hardware interrupts with the code `self.sensor_left.when_activated = self.count_left` and `self.sensor_right.when_activated = self.count_right`. This means the instant an encoder slot passes through the sensor and triggers a high voltage, the Raspberry Pi's hardware interrupts the program to run the counting function (`count_left` and `count_right` respectively), ensuring no missed ticks and greater efficiency compared to the method of polling.
- Counting variables: We make use of two variables for each of the wheels - one is an accumulative counter and another is a directional value. Both of them are set to 0 during initialization.

2) *getdirection function*: This function takes the values of `angular` and `linear` from `cmd_vel` to assign values for the direction variables (`self.dleft` and `self.dright`).

- If commanded to move forward (`linear > 0`), both left and right directions are set to 1.
- If commanded to move backward (`linear < 0`), both directions are set to -1.

- If turning in place to the left (angular >0), the left wheel moves backward (-1) and the right wheel moves forward (1).
- If turning to the right (angular <0), the left moves forward (1) and the right moves backward (-1).
- Otherwise, the direction variables are set to 0.

3) **count_left** and **count_right** functions: When the hardware interrupts trigger **count_left** or **count_right**, the code simply adds the current direction variable (**self.dleft** or **self.dright**) to the running total of ticks (**self.left_ticks** or **self.right_ticks**).

F. Optical_Node.py

This node utilizes a custom python package provided by the hardware maker PIMORONI [6]. After performing initialization for the required pins, the node created an infinite loop where the node repetitively requests the values of the horizontal and vertical pixel shift respectively. For transmission efficiency, a *TwistStamped* with preset reference coordinates frame and timestamp wraps the two mentioned values. The package is then published.

1) Initialization:

- Individual GPIO pins declaration is not required for this function, as the sensor communicates using Serial Peripheral Interface protocol (SPI), rather than sending raw high/low electrical pulses.
- Publisher: Sends out *TwistStamped* messages on a topic named **optical_flow** with a queue size of 10.

2) **timer_callback** function: the function attempts to read the **delta_x** and **delta_y** values using the sensor's built-in **get_motion()** function. In this case, the two delta values represent the raw optical flow counts (pixels) per 0.1-second-intervals, then add them to the running **self.x** and **self.y** totals. These accumulated totals are cast as floats and stored in the **linear.x** and **linear.y** fields of the message. All angular velocities and the linear Z velocity are set to 0.0, as this sensor only reads 2D planar movement.

G. Fusion_Node.py

This ROS2 Python node, performs sensor fusion from the data gathered from the encoder node (wheel encoders) and the optical node (optical flow sensor). The primary goal of this node is to calculate the robot's current pose (position and orientation) in a 2D space and publish it for other parts of the ROS system to use.

1) **Initialization**: In the initialization function, we first declare the variables that will hold the processed data:

```
# inheritance variables
self.x = 0.0
self.y = 0.0
self.left_ticks = 0
self.right_ticks = 0
self.yaw = 0.0
```

Fig. 14. Parameters initialization

In which **x** and **y** will hold the robot's position, accumulated ticks for each wheels are stored in **left_ticks** and **right_ticks** variables. The yaw angle of the chassis is calculated and stored in the **yaw** variable.

Many equations in this function require us to manually measure certain characteristics of the chassis. These include the wheels' diameter, spacing between two wheels, number of slots on the encoder disk, the distance covered per optical encoder tick and the offset of the optical sensor. All of these values is then inserted inside the function as constants:

```
#vehicle information
self.wheel_diameter = 0.071
self.wheel_track = 0.142
self.ticks_per_rev = 20
self.meters_per_count = 1.0 / 29750.0

# distance from optical sensor to the rear axle [m] (because the optical is not centered)
self.sensor_offset = 0.075
```

Fig. 15. Chassis Characteristics

Following the equations described in the previous section, we can calculate the rotation of the robot though the radial difference between the wheel encoders:

```
self.dist_per_tick = (math.pi * self.wheel_diameter / self.ticks_per_rev)
self.rad_per_tick_diff = (self.dist_per_tick / self.wheel_track)
```

Fig. 16. Robot rotation calculation

Finally, we declare the subscribers and publishers of this node. We listen for the **ticks_left**, **ticks_right** and also the **optical_flow** topics from the sensor nodes, while package our calculation result inside a *Twist* message and send it through a topic named **processed_data**. For this, we made use of the three callback functions.

2) **flow_callback** function: This function is triggered every time a new message arrives from the optical flow sensor. It handles translational movement (X and Y). The pipeline of the function can be describe as follows:

- **Conversion to Meters**: It extracts the raw X and Y velocities (counts) from **msg.twist.linear.x** and **msg.twist.linear.y**, multiplying them by **self.meters_per_count** to get the real-world distance traveled (**delta_x_real** and **delta_y_real**) since the last reading.
- **Coordinate Transformation**: It calculates the new global **self.x** and **self.y** position using a 2D rotation matrix based on the robot's current heading (**self.yaw**)

```
def flow_callback(self, msg):
    delta_x_real = msg.twist.linear.x * self.meters_per_count
    delta_y_real = msg.twist.linear.y * self.meters_per_count

    self.delta_x = msg.twist.linear.x
    self.delta_y = msg.twist.linear.y

    self.x += (delta_x_real * math.sin(self.yaw) + delta_y_real * math.cos(self.yaw))
    self.y += (- delta_x_real * math.cos(self.yaw) + delta_y_real * math.sin(self.yaw))
```

Fig. 17. Flow_callback function

3) *left_callback* and *right_callback* functions: These functions update the robot's absolute heading (yaw) based on differential drive kinematics. The following steps are triggered when new encoder data arrives for the left and right wheels, respectively.

- Store the incoming tick count in *self.left_ticks* or *self.right_ticks*. Since the tick is cumulated in the encoder node before sending to fusion node, for case that the wheel still spinning but the robot is stuck in a position, we need a way to eliminate the cumulated part when the wheel is still spinning but the robot is not moving in x, y position. That's why offset values (*offset_left* and *offset_right*) are used. When the optical flow sensor detects significant movement (both absolute value of *delta_x* and *delta_y* exceed the predefined threshold, in this case is 3), the actual wheel ticks (*left_ticks*, *right_ticks*) are recalculated by subtracting the offset (*offset_left*, *offset_right*) from the received encoder ticks (*msg*). If no significant movement detected, this offset is updated to match those cumulated ticks that are being cumulated while the vehicle is standing still, preventing unwanted changes in orientation when the robot is moving again after a period of stationary with wheel still spinning.
- Immediately recalculate *self.yaw*. Because differential drive robots rotate by spinning their wheels at different speeds (or tracking different total distances), the heading is derived directly from the difference in wheel travel, as describe in the previous section. The tick difference is then multiplied by the conversion factor *rad_per_tick_diff*, which converts the encoder tick difference into an angular displacement in radians.

```
def left_callback(self, msg):
    if (abs(self.delta_x) > 3 and abs(self.delta_y) > 3): #condition for cumulative ticks
        self.left_ticks = msg.data - self.offset_left
    else:
        self.offset_left = msg.data - self.left_ticks
    self.yaw = self.rad_per_tick_diff * (self.right_ticks - self.left_ticks)

def right_callback(self, msg):
    if (abs(self.delta_x) > 3 and abs(self.delta_y) > 3): #condition for cumulative ticks
        self.right_ticks = msg.data - self.offset_right
    else:
        self.offset_right = msg.data - self.right_ticks
    self.yaw = self.rad_per_tick_diff * (self.right_ticks - self.left_ticks)
```

Fig. 18. Left and Right callback functions

4) *compute* function: This is the timer callback that runs every 0.1 seconds. It acts as the final step of the fusion process, applying the physical offset correction and publishing the result.

The *self.x* and *self.y* variables track the position of the optical sensor. To find the position of the robot's center of rotation (the rear axle), it subtracts the *sensor_offset* using trigonometry. Finally, it assigns the calculated values to the message, publishes the *msg* to the *processed_data* topic, and print out a string to the console displaying the X, Y, Wheel Ticks, Yaw (in radians), and Yaw (in degrees) for debugging

```
def compute(self):
    msg = Twist()

    msg.linear.x = (self.x - self.sensor_offset * math.cos(self.yaw))
    msg.linear.y = (self.y - self.sensor_offset * math.sin(self.yaw))
    msg.angular.z = self.yaw

    self.publisher_.publish(msg)
```

Fig. 19. compute function (get_logger function not included)

H. Odometry_Node.py

This Python program is used to compile a ROS2 node that listens to robot movement data and visualizes its 2D position and orientation (heading) in real-time, making use of the matplotlib library.

```
def __init__(self):
    super().__init__('odometry_node')

    # Current robot pose
    self.x = 0.0
    self.y = 0.0
    self.yaw = 0.0
    self.yaw_deg = 0.0

    # Subscribe to Fusion Node output
    self.create_subscription(
        Twist,
        'processed_data',
        self.odom_callback,
        10
    )

    # Update graph every 100 ms
    self.timer = self.create_timer(
        0.1,
        self.update_plot
    )

    # Create graph window
    plt.ion()
    self.fig, self.ax = plt.subplots()

    self.get_logger().info("Odometry Node Started")
```

Fig. 20. Initialization function of the Odometry Node

1) Initialization:

- Subscription: It creates a subscription to a topic named 'processed_data', expecting messages of type Twist. Incoming messages are routed to the *odom_callback* function.
- State Variables: It creates variables to track the robot's pose: *self.x*, *self.y*, *self.yaw* (in radians), and *self.yaw_deg* (in degrees), setting them all to 0.0 initially. They will later take the values that arrived from the fusion node.
- Plot Setup: It turns on interactive mode for matplotlib using *plt.ion()* and initializes the figure and axes (*self.fig*, *self.ax*) for live drawing.

2) *odom_callback* function: This function is the callback triggered every time a new Twist message is published to the *processed_data* topic.

- Position Update: It reads *msg.linear.x* and *msg.linear.y* to update the robot's current x and y coordinates.
- Heading Update: It reads *msg.angular.z* to update the robot's yaw angle in radians.
- Normalization: It calculates *self.yaw_deg* by converting the radians to degrees and using the modulo operator (mod 360) to ensure the angle stays normalized within the 0 to 360-degree range.

3) *update_plot* function: This is the most important function inside this node. It handles the visual rendering of the robot's state and is called repeatedly by the timer.

- Clearing the Canvas: It clears the axes (*self.ax.clear()*) to prevent older positions from cluttering the graph, allowing only the current state to be drawn.
- Drawing the Robot: It plots the robot's current x and y coordinates as a dot.
- Drawing the Heading Arrow: It calculates the directional vector for the robot's heading. Using a predefined *arrow_length* of 0.6, it applies trigonometry to find the directional components.

$$dx = 0.6 \cdot \cos(\text{yaw}) \quad (12)$$

$$dy = 0.6 \cdot \sin(\text{yaw}) \quad (13)$$

It then uses *self.ax.arrow* to draw an arrow from the robot's position to show which way it is facing.

- Formatting: It sets the title, labels, and enables a grid. It locks the X and Y axes limits between -2 and 2 meters so the plot window remains stable rather than resizing dynamically. It also enforces an 'equal' aspect ratio so the arrow doesn't look distorted.
- Rendering: It uses *plt.pause(\$0.02\$)* to temporarily yield execution to the GUI event loop, allowing the plot to actually render on the screen. It logs the currently calculated X, Y, and Yaw (in degrees) to the ROS console.

VII. RESULT AND TESTING

A. Experimental Setup and Methodology

Testing was conducted on a smooth, matte indoor floor to evaluate the odometry system under typical operating conditions. The testing conditions ensured the distance between the operator's machine and the chassis never exceeded 10m for signal stability.

The battery was fully charged with each cell showing 4.2V. Everytime the robot is powered on, a time delay is required in order for the Raspberry Pi onboard operating system to finish booting and established a connection to the internet hotspot of the host machine.

After getting a connection to the Raspberry Pi, we startup all of the nodes on our system using the terminal and SSH. We then started sending the directive commands using 'W', 'A', 'S', 'D' keys and observe the result. The robot's response to

```
self.ax.set_title("Robot Odometry")
self.ax.set_xlabel("X Position (m)")
self.ax.set_ylabel("Y Position (m)")
self.ax.grid(True)

# Keep robot centered in view
self.ax.set_xlim(-2, 2)
self.ax.set_ylim(-2, 2)

self.ax.set_aspect('equal')

plt.pause(0.02)

self.get_logger().info(
    f"X={self.x:.3f} m, "
    f"Y={self.y:.3f} m, "
    f"Yaw={self.yaw_deg:.1f} deg"
)
```

Fig. 21. Plot creation and data logging

```
self.ax.clear()

# Draw current robot position
self.ax.plot(
    self.x,
    self.y,
    'o',
    markersize=5
)

# Draw heading vector
arrow_length = 0.6

# Robot convention:
# WORLD Forward = +X
# WORLD LEFT = +Y
dx = arrow_length * math.cos(self.yaw)
dy = arrow_length * math.sin(self.yaw)

self.ax.arrow(
    self.x,
    self.y,
    dx,
    dy,
    head_width=0.15,
    length_includes_head=True
)
```

Fig. 22. Drawing of the robot's representation

abnormalities was tested by driving it into a wall and let the motors continue spinning.

B. Observation

- The robot correctly followed the directive commands. Apart from the delay introduced by our implementation

of the controls, the system experience little to no input lag.

- When instructed the robot to move on a straight line, a notable amount of drift were observed. This drift is largely based on the prior orientation of the caster wheel as well as the remaining power on the battery.
- When attempted to perform continuous and stationary turning, we also experience an error in the reported yaw angle compared to the actual orientation. This error accumulated though time and eventually results in an error of up to 90 degrees for 10 seconds of turning.
- The errors in both current coordinates and orientation get transfered to the graphical plot and required a reset of the entire network to remove the accumulative error.
- When the chassis was stopped by the wall but the motors were still spinning, the cross-checking code worked as intended by discarding the reading on the encoder disk and stopping updates in coordinates to the graph.

VIII. DISCUSSION AND OUTLOOK

A. Performance Discussion and System Evaluation

The experimental results demonstrate both the architectural validity and the physical constraints of the implemented ROS2 odometry framework. The system successfully achieved its baseline goals of wireless remote control, multi-node data transmission, and real-time tracking. However, a detailed analysis of the system components reveals critical areas of performance variance:

- **Control Interface and Latency:** Bypassing the operating system's keystream hold limitation using a timed non-blocking select mechanism allowed for sustained continuous motion commands. However, ignoring the first few inputs forced a trade-off, introducing a noticeable operational delay from input to execution that limits rapid maneuverability.
- **Mechanical Anomalies and Drift:** Driving in a straight line revealed persistent sideways drift. Although partially mitigated by applying a 23% software power reduction factor to the right motor within the motor node initialization, the residual drift highlights inherent physical limitations. This behavior is dictated by the starting orientation of the passive caster wheel and fluctuations in battery voltage levels.
- **Kinematic and Accumulative Rotation Errors:** The most severe error occurred during continuous, stationary turning. The open-loop differential kinematics calculations accumulated difference of up to 90 degrees over a 10-second turning interval. Because these positional and orientation are continually integrated into the current coordinates, the live canvas map becomes distorted over time, necessitating a complete reset of the ROS2 network nodes to re-zero the origin.
- **Data Fusion and Abnormality Tracking:** The implementation of custom sensor cross-correction logic inside the fusion node proved highly robust. When the vehicle collided

with a solid wall barrier, the algorithm properly identified the discrepancy between updating wheel encoders and static optical flow displacement. Discarding the erroneous encoder data completely halted improper graph position shifts, proving the utility of dual-sensor setups under high wheel-slip conditions.

B. Technical Outlook and Future Enhancements

To evolve this prototype into a more dependable automotive simulation platform, several hardware and software iterations are planned:

- **PID Controller Integration:** One way to reduce the sideways drift is the implementation of a PID controller into the motor node. The current design uses a fixed value for motors PWM modulation. While being more simple to implement and solved the initial motor power mismatch problem, the manual tuning property is not viable for variable power from the battery's power state and more complex environmental properties. The introduction of a feedback element helps the machine better adapt to these challenges.
- **Chassis Steering Optimization:** Replacing the high-resistance passive caster wheel with a low-friction omnidirectional ball caster would eliminate the mechanical pulling forces that induce sideways drift during forward travel.
- **Control Node Overhaul:** The teleoperation latency caused by keyboard processing streams can be resolved by transitioning the control node input mechanism away from standard console terminal keystrokes. Integrating a physical gamepad controller node using standard Linux joystick drivers would remove the input filtering delay entirely, yielding instant motor driver response.

IX. TEAM EXPERIENCE AND CONTRIBUTION

The project was collaboration of two team members, with responsibilities divided from the hardware implementation to the software development on the Ubuntu Virtual Machine as well as on the Raspberry Pi. This allowed both members to focus on different aspects of the odometry robot while maintaining continuous work flow during the integration and testing phases. Regular discussions were made throughout the project to ensure compatibility between the software and hardware components and to resolve issues encountered during system integration, testing.

Person A (Quoc An Nguyen)

Person A primarily responsible for the hardware implementation and the development of the motor_node, optical_node, and encoder_node on the Raspberry Pi 4:

- Installing ROS2 on the Raspberry Pi 4.
- Developing the motor, encoder, and optical flow nodes.
- Hardware assembly, wiring, and component mounting.

Person B (Viet Hung Tran)

Person B primarily responsible for the fusion_node, odometry_node on the Raspberry Pi and control_node on the Ubuntu Virtual Machine.

- Installing ROS2 on the Ubuntu Virtual Machine.
- Setting up the ROS2 communication network between the host machine and the Raspberry Pi.
- Developing the control, fusion, and odometry nodes.

Collaborative Contributions

At the beginning phase of the project, both members participated in defining the project objectives, analysing the system requirements, and designing the overall ROS2 architecture. Together, the team determined the required nodes, topics, and message flow between nodes on the Ubuntu Virtual Machine and on the Raspberry Pi.

Although tasks were divided according to the project plan, both team members actively participated in reviewing each other's work throughout the development phase to identify implementation errors, verify the correctness of the algorithms, and refine the software logic.

Finally, both members contributed to preparing the project documentation, recording experimental results, and finalize the project. These effort ensured that all project objectives were completed and the robotic system is operated as expected.

X. MAIN LEARNING OUTCOME

The development and deployment of this odometry-tracking robot chassis provided substantial practical experience in modern automotive and robotics engineering concepts. By transitioning from theoretical designs to a physical prototype, the following key learning outcomes were achieved:

- **ROS2 Architecture and Distributed Systems:** Gained comprehensive experience in configuring and deploying a Robot Operating System 2 (Humble) network across multiple operating systems (Ubuntu 22.04 natively and via Virtual Machine). This included designing independent software nodes, establishing SSH network communication, and managing real-time data streams utilizing publishers, subscribers, and standardized message types (Twist, TwistStamped).
- **Hardware-Software Integration:** Developed proficiency in interfacing high-level Python software with physical low-level hardware components. This involved configuring the Raspberry Pi's GPIO pins, utilizing hardware interrupts for precise IR encoder counting, communicating with the PMW3901 optical flow sensor via SPI protocols, and applying Pulse Width Modulation (PWM) through an L298N driver to regulate DC motor speeds.
- **Applied Kinematics and Coordinate Mathematics:** Bridged the gap between theoretical math and physical behavior by translating raw sensor data into actionable spatial awareness. This required the practical application of differential drive kinematics, trigonometric coordinate transformations, and rotational matrices to continuously map discrete sensor ticks and pixel shifts into a global 2D representation of the robot's pose.
- **Sensor Fusion and Fault Tolerance:** Learned the critical importance of redundancy and cross-validation in real-world environments. Implementing custom logic to actively compare data streams—specifically overriding

faulty wheel encoder readings with optical sensor data when the chassis was physically stalled—provided first-hand insight into mitigating hardware limitations and managing accumulative error.

- **Real-time Telemetry and Visualization:** Acquired hands-on skills in processing and visualizing live robotic telemetry data. By utilizing Python's matplotlib library to render interactive, continually updating vector graphics, the system successfully translated complex numerical matrices into an intuitive graphical interface for operator observation and system debugging.

ACKNOWLEDGMENT

The completion of this robot odometry project would not have been possible without the support and resources of several individuals and institutions. We would like to express our gratitude to our supervisors Prof. Ansgar Meroth and Mr. Pascal Keller, for their continuous guidance and technical feedback throughout the Electronic Systems Lab course.

REFERENCES

- [1] Open Robotics, "ROS 2 Documentation". [Online]. Available: <https://docs.ros.org/en/humble/>. [Accessed: Jul. 18, 2026].
- [2] John Olatubosun, "Explaining ROS 2 to Software Engineers," Medium, Nov. 13, 2025. [Online]. Available: <https://medium.com/codex/explaining-ros-2-to-software-engineers-eed7afdcc1d8>. [Accessed: Jul. 18, 2026].
- [3] Husarion, "ROS 2 Introduction". [Online]. Available: <https://husarion.com/tutorials/ros2-tutorials/1-ros2-introduction/>. [Accessed: Jul. 18, 2026].
- [4] Random Nerd Tutorials, "ESP32 with DC Motor and L298N Motor Driver – Control Speed and Direction;". [Online]. Available: <https://randomnerdtutorials.com/esp32-dc-motor-l298n-motor-driver-control-speed-direction/>. [Accessed: Jul. 18, 2026].
- [5] Jobit Joseph, "Interfacing PMW3901 Optical Flow Sensor With ESP32," Circuit Digest, Apr. 3, 2025. [Online]. Available: <https://circuitdigest.com/microcontroller-projects/interfacing-pmw3901-optical-flow-sensor-with-esp32>. [Accessed: Jul. 18, 2026].
- [6] Philip Howard, " pmw3901-python," Github. [Online]. Available: <https://github.com/pimoroni/pmw3901-python>. [Accessed: Jul. 18, 2026].
- [7] JZIK, "24V / 12V To 5V 5A Power Buck Module DC-DC Step Down Power Supply Converter with LED;". [Online]. Available: <https://www.amazon.com/example-link-to-product>. [Accessed: Jul. 19, 2026].
- [8] Ebay, "10Pcs Coded Disc Encoder Robot Speed Test 20Holds Motor Speed Sensor For Arduino," 2026. [Online]. Available: <https://www.ebay.com/itm/225524619081>. [Accessed: Jul. 19, 2026].
- [9] Amazon, "LM393 Comparator Speed Sensor Detection Module LM393 Chip Slot Motor Measurement Compatible with MCU ARM Arduino," 2026. [Online]. Available: <https://www.amazon.de/-/en/HALJIA-LM393-Comparator-Measurement-Compatible/dp/B06XDMCPNG>. [Accessed: Jul. 19, 2026].